

# permfuse: a Policy Layer over Linux OverlayFS

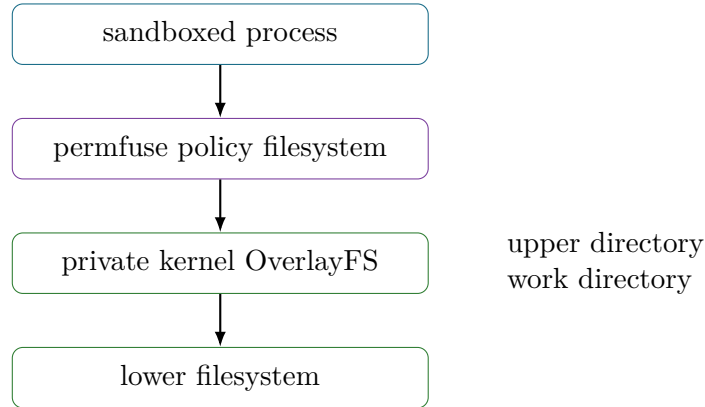
permbox-fuse3

July 30, 2026

## Abstract

permfuse is a library that presents a filesystem through a dynamic path policy. Linux OverlayFS stores all changes. A low-level FUSE filesystem controls namespace visibility and decides whether each path is read-only, writable, or requires an interactive decision. This split keeps filesystem mechanics in the kernel and keeps the policy implementation small.

## 1 Stack



OverlayFS performs copy-up, whiteouts, opaque directory handling, sparse file I/O, mmap, hard links, and rename. The FUSE layer does not reproduce those operations. It checks policy and forwards permitted calls to the merged mount.

## 2 Policy

Rules are stored in a memory-mapped radix trie. Lookup returns the nearest explicit slash-component ancestor.

| Rule            | Result                                        |
|-----------------|-----------------------------------------------|
| <b>whiteout</b> | the path appears not to exist                 |
| <b>r</b>        | metadata and read-only opens                  |
| <b>rw</b>       | reads and mutations may continue              |
| <b>ask</b>      | obtain and store a decision before continuing |

The zero trie value is reserved for an internal node without an explicit rule. Durable rules therefore use nonzero values.

An ask operation checks the trie a second time before prompting. Prompt locks are striped by path hash, so two requests for one path do not create duplicate questions while unrelated paths can continue independently. The returned decision is written as an explicit rule before the original operation opens a handle.

### 3 Unix permissions

The FUSE mount uses `default_permissions` and returns ownership and mode metadata from the merged OverlayFS object. The kernel checks the caller's UID, GID, mode bits, ACL, sticky directory rules, and execute permission. `permfuse` then applies its four-state path policy. OverlayFS finally performs the operation using its mount credentials.

This arrangement avoids a partial userspace implementation of supplementary groups, ACLs, capabilities, and sticky directories.

### 4 Open handles and passthrough

Policy is resolved before opening a handle. A handle stores one access byte. Read, write, flush, `fsync`, and release do not lock or read the trie. Later policy changes affect later operations and opens, not an existing resolved descriptor.

When supported, the daemon registers the merged OverlayFS descriptor with FUSE passthrough. Kernel read, write, splice, and mmap can then use that descriptor directly. A read-only rule receives a read-only descriptor. A writable rule receives the flags requested by the caller. The normal FUSE I/O path is used when passthrough is unavailable.

### 5 Concurrency

The inode map lock covers only publication and reference counts. No backing filesystem syscall runs while that lock is held. An inode is destroyed only after its FUSE lookup count and open-handle count both reach zero. This makes concurrent forget and release operations safe.

The policy trie uses a `std::Io.RwLock`. Individual lookups hold its read side. Rule batches and ask decisions hold its write side for their complete transaction.

### 6 Session layout

A session directory contains:

```
session/  
  lock  
  upper/  
  work/  
  merged/
```

The lock prevents two drivers from mounting or applying one session. Upper and work are on the same filesystem. The initial mount requests `redirect_dir=on`, `index=on`, `xino=auto`, and `metacopy=off`. If inode indexing is unsupported, the mount retries without it and logs the result.

### 7 Apply and recovery

Apply is allowed only after both mounts have stopped. It walks the upper tree. Regular files are copied into a destination-side temporary file. The temporary file is synced and renamed over its destination. The destination directory is then synced. Only after those steps does apply remove the upper entry.

Whiteouts delete the corresponding lower object. Opaque directories replace the lower directory contents. Symlinks and special files are recreated. OverlayFS bookkeeping attributes are not copied into the lower tree.

The upper tree is the checkpoint. A crash before upper removal repeats an idempotent publication. A crash after upper removal has already completed that operation. A bounded apply leaves remaining upper entries for the next call. Discard removes one selected upper subtree.

## 8 Limits

Editing an OverlayFS lower tree while it is mounted is unsupported by Linux. permfuse does not attempt to make it safe. Kernel OverlayFS and FUSE support, mount privileges, upper filesystem xattrs, and valid directory entry types are required. Passthrough additionally needs kernel support and `CAP_SYS_ADMIN`.